Ricardo D. Solis

778-861-6944 | rds11@sfu.ca | linkedin.com/in/ricardo-solis-9a9068237 | github.com/rsolis096

Introduction

This portfolio highlights my professional and personal work in game development, with a focus on gameplay programming, multiplayer systems, technical debugging, and real-time graphics. I provide additional context on the projects I have developed or contributed to beyond what is included in my resume. All my personal projects are available on my GitHub github.com/rsolis096.

Professional Projects

Without A Hitch

Company: Codename Entertainment
Role: Junior Game Developer (C#, Unity)
Steam Page: `https://store.steampowered.com/app/4043720/Without_A_Hitch/`

At Codename Entertainment, I worked as a Junior Game Developer on the multiplayer cooperative game *Without A Hitch* (WOAH). I joined WOAH after completing my work on Codename Entertainment's *Project Deathspike* in Unreal Engine 5. This required moving from Unreal Engine to Unity and C#, a toolset I had not previously used professionally. By tracing existing systems, asking focused technical questions, and taking ownership of complex features, I quickly became a productive member of the development team.

I contributed to WOAH from an early prototype through release and post-release support. My largest systems included a reusable networked statistics and achievement architecture with Steam and Epic integration, and a server-authoritative Mine enemy state machine that synchronized behaviour, physics-based movement, attacks, audio, and procedural animation through Mirror. I also worked extensively on menus, HUD elements, notifications, contextual prompts, full-screen map interaction, diagnostic tools, localization, and Steam Deck-specific UI.

Following launch, I continued supporting WOAH through multiplayer debugging, player-feedback improvements, and art and audio integration. I converted several project-specific systems into reusable framework components, particularly around networking, player state, UI, statistics, achievements, and persistence. That work later became part of the technical foundation used to begin *Folded Fables*.

My Contributions:

1. Statistics and Achievement System
    - Designed and implemented a networked player statistics system to track gameplay events, player progress, and session-based performance.
    - Integrated the statistics system with Steam and Epic Online Services achievements, allowing player actions to update platform-specific achievement progress and unlocks.
    - Built player-facing statistics and achievement UI, giving players clear feedback on progress, completed goals, and gameplay milestones.
    - Structured the system around reusable stat definitions, allowing other developers to add new tracked stats and achievements with minimal setup.
    - Implemented cloud-based stat persistence using Steam and Epic cloud saves, with local save batching to store data before synchronizing with platform services.
    - Integrated stat tracking across gameplay systems and progression events, handling local, global, and per-player updates depending on each achievement requirement.

- Debugged synchronization and persistence issues across local gameplay, networked sessions, Steam, and Epic services to ensure stats and achievements updated reliably.
- Implemented achievement logic ranging from simple cumulative stats, such as distance driven, to complex run-based conditions, such as completing the game under specific constraints.
- Refactored the statistics and achievement system into reusable framework components that could be carried into future multiplayer Unity projects.

2. Enemy Mine State Machine

- Independently designed and implemented a fully networked enemy Mine state machine in C#.
- Built server-authoritative AI behaviour using Mirror, synchronizing state transitions, movement, attack timing, audio cues, and gameplay feedback across multiplayer sessions.
- Integrated physics-based movement and procedural animation systems using rigid bodies, allowing the Mine to move believably while remaining responsive for all players.
- Developed a state-driven audio system that allowed audio designers to assign clips per behaviour state, including playback timing, transitions, and attenuation controls.
- Structured the Mine system to support reusable variations and modules, enabling the same core behaviour framework to be adapted for other gameplay features.
- Iterated on the enemy through testing and feedback, with the final Mine behaviour receiving very positive responses from players and supervisors.

3. Gameplay User Interface

- Built statistics and achievements UI for Steam and Epic Online Services, with an admin/debug interface for testing stat updates and achievement triggers.
- Enhanced the minimap system with compute shaders, supporting full-screen map interaction including zooming, panning, player tracking, and UI markers.
- Implemented context-sensitive HUD controls that update dynamically to show only the inputs relevant to the player's current gameplay state.
- Created animated UI components for player feedback, including event-driven popups, unlock notifications, and discovery messages that respond to gameplay events.
- Localized most in-game UI text for multiple languages, maintaining readable layouts through text scaling and formatting.
- Implemented Steam Deck-specific UI behaviour using the Steamworks SDK to detect the device and display appropriate interactions.

4. Art and Audio Integration

- Integrated the majority of in-game art and audio assets provided by our art and audio teams, ensuring content worked correctly within gameplay, UI, and multiplayer systems.
- Implemented in-game points of interest, updated models, and visual effects connecting these assets to networked interactive gameplay behaviour where needed.
- Built networked audio playback systems that synchronized sound effects across multiplayer sessions while exposing a simple workflow for the audio engineer to assign, tune, and test clips.
- Networked art-driven gameplay elements that required animation, player interaction, state changes, or synchronized visual changes across clients.
- Collaborated with artists and our audio engineer to debug implementation issues and ensure assets matched gameplay requirements in production.

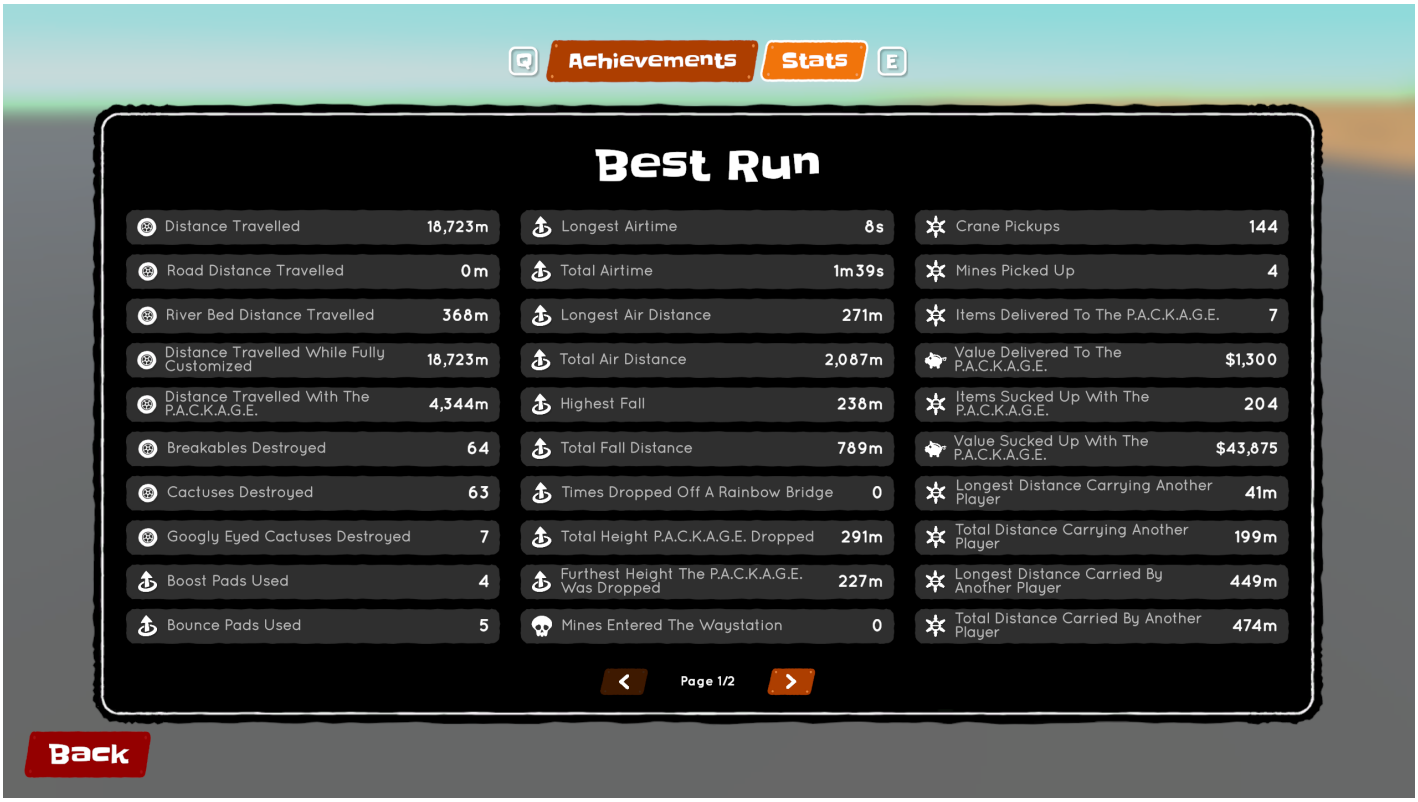
5. Multiplayer Testing and Debugging

- Investigated and resolved multiplayer defects involving server authority, synchronized state, physics interactions, persistent data, and differences between host and client behaviour.
- Created dedicated test scenes and debugging tools to reproduce multiplayer issues and verify fixes across host and client sessions.
- Investigated and fixed timing-sensitive issues involving reconnecting players, persistent data, networked audio, and host-client state differences.

Screenshots:



Achievements UI



Stats UI

Screenshots:



Big Mine enemy variation



Small Mine enemy variation

Professional Projects

Folded Fables: Tidy Up the Dragon Hoard

Company: Codename Entertainment
Role: Junior Game Developer (C#, Unity)
Steam Page: store.steampowered.com/app/4995030

Folded Fables is an online cooperative organization game built in Unity. I worked on the project throughout its development, from the initial prototypes until shortly before release. My first major responsibility was creating the project's starting point from the existing WOAH codebase. I removed WOAH-specific code, scenes, content, and dependencies while preserving and adapting the important networking, UI, input, and player-controller systems. This gave the team a clean, functional multiplayer project that could be developed immediately instead of rebuilding its technical foundation from scratch.

From that foundation, I contributed to the core item-organization gameplay: items, player inventories, containers, fixtures, placement validation, interaction feedback, and content-authoring tools. I also developed multiplayer abilities and progression, including reconnect-safe player data and global upgrades that remained correct for late-joining players. My work extended into controller rebinding and navigation, statistics and achievements, dragon-pedestal progression, localized signage, networked audio, and production polish.

My Contributions:

1. Project Foundation and Shared Framework
    - Removed WOAH-specific gameplay, content, scenes, and dependencies while keeping the existing project stable and playable.
    - Preserved and adapted reusable Mirror networking, multiplayer-session, UI, input, and player-controller systems so the team could begin feature development from a clean working baseline.
    - Continued separating shared systems from game-specific behaviour, making the framework easier to extend across a second multiplayer Unity production.
2. Item Organization and Interaction Systems
    - Developed core systems for items, inventories, containers, fixtures, signs, and placement rules that support the game's central organization mechanics.
    - Implemented server-validated item interactions, placement state, spacing, physics behaviour, visual feedback, and safeguards for invalid item or container states.
    - Built editor workflows for defining item and fixture sets, supporting efficient integration of the game's large catalogue of objects.
3. Multiplayer Abilities and Persistent Progression
    - Implemented active and passive abilities including teleportation, double jump, item pulling and throwing, increased carrying capacity, and fixture highlighting.
    - Developed the multiplayer Shelf Sense ability, which validates compatible items and containers on the server and queues automatic placement for participating players.
    - Made abilities, global upgrades, carrying capacity, and related UI reconnect-safe so progression was restored for disconnected and late-joining players.
    - Contributed to ability trees, cooldowns, unlocks, dragon-pedestal progression, and the supporting player-facing and administrative UI.
4. Statistics, Achievements, Controls, and Production Polish
    - Adapted and expanded the reusable statistics and achievement framework for shop results, completed departments, abilities, caught items, dragon eggs, and other gameplay events.

- Restored and expanded keyboard and controller rebinding, input icons, menu navigation, and settings behaviour.
- Integrated networked audio, localized fixture signage, placement effects, container-tag visuals, and other feedback used throughout production.

Screenshots:



Folded Fables: Tidy Up the Dragon Hoard



The dragon hoard's multiplayer item-organization environment

Professional Projects

Project Deathspike – Unreleased First-Person Horror Game

Company: Codename Entertainment
Role: Junior Game Developer (C++, Unreal Engine 5)

Project Deathspike is an unreleased first person horror game I contributed to at Codename Entertainment. I worked in Unreal Engine 5 using Blueprints and C++ to develop key gameplay features, fix bugs, and implement performance fixes. I used Visual Studio as my primary development environment, relying on its debugger to investigate defects and Unreal Insights to analyze engine performance. During development of *Deathspike*, much of the team transitioned to *Without A Hitch*, leaving me as the primary full-time developer continuing work on the project. During this time, I worked directly with artists and designers to implement their work into the game as well as continuing work on key features like the player upgrade systems and performance improvements. I worked mostly unsupervised and would reach out to developers on other projects when I needed help; however, I was able to adapt to this environment and contributed quality work that received excellent feedback from my supervisor and designers. Most of my original contributions were implemented in Blueprints, although I also used C++ when extending or repurposing existing systems.

Before switching over to *Without A Hitch*, I was tasked with finding causes of performance issues in the main level and optimizing them where possible. This led me to use Unreal Insights as well as stat commands to get a more in-depth description of the processes taking up the most compute time. Through this work, I optimized lighting, interactables, and skeletal-mesh physics operations. These changes improved stability and produced frame-rate gains of up to 100% in some areas, with noticeable improvements throughout the level. The results received very positive feedback from my supervisor. This work deepened my understanding of Unreal Engine technologies such as Lumen, tick optimization, and object culling. I carried these lessons into later feature development on *Without A Hitch*.

Due to the conditions of *Project Deathspike's* development, I am limited in the visuals I can share, but I can discuss my contributions and the technology behind them.

My Contributions:

1. Performance Optimizations
    - Profiled runtime performance using Unreal Insights and Stat commands, recording and analyzing gameplay sessions across varied level locations to uncover repeated performance issues and scenario-specific hot spots.
    - Improved performance by optimizing animated physics-driven static meshes such as swinging chains and lanterns, disabling updates when outside the player's view to reduce unnecessary simulation cost.
    - Leveraged Unreal Engine lighting visualization tools to identify performance hot spots. I optimized attenuation, draw distance, and shadow settings while replacing low-impact point lights with spotlights where appropriate, achieving performance improvements of up to 100% in some areas.
2. Scripted Events (Action On Event)
    - Utilized a custom, in-house scripting tool called "Action On Event" to script horror events. Some events I scripted include a chandelier shaking as the ground trembles when the main monster is alerted to the player's presence, parts of a suspension bridge breaking as the player crosses it, and a monster attacking an NPC.
    - Utilized Action On Event to support dialogue that changed based on player choices and actions.
3. User Interface
    - Developed multiple gameplay UI features from scratch and replaced debug log and text based feedback with player facing custom UI, improving usability and in-game clarity.

- Implemented a reusable UI system for a shrine-based player upgrade mechanic, integrating inventory items to apply upgrades and dynamically reflect upgrade state in the UI.
- Built a modular item interaction UI system supporting pickupable items and their usage, with contextual feedback for using and crafting items while adhering to the art team's requirements.

Professional Projects

Big Country – Dog Companion Feature

Company: Tidepool Games (in partnership with The Centre for Digital Media)
Role: Game Developer (C++, Unreal Engine 5)
Project Site: https://thecdm.ca/projects/big-country-tidepool-games

As part of a client-driven project at the Centre for Digital Media with Tidepool Games, I contributed to Big Country, an upcoming open-world exploration game built in Unreal Engine 5. I collaborated with animators, product managers, developers, and designers to create a dog companion named Otter. Otter is designed to be both a playful sidekick and a practical utility for the player. The prototype was very well received and has appeared in private trailer showings and investor demos at conferences such as GDC. Unfortunately, strict confidentiality requirements mean I retain only limited materials and no longer have access to the source code. For more information on the project, visit https://thecdm.ca/projects/big-country-tidepool-games. Please note that the trailer linked there was produced and directed by our team and may not fully reflect Tidepool's final vision.

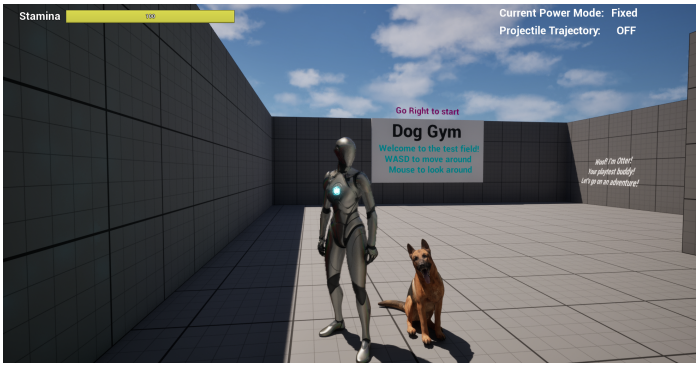
My Contributions:

1. AI Behaviour Trees & C++/Blueprints Integration
    - Contributed to and implemented parts of Otter's core behaviours including Follow, Stay, Fetch, Pet, and Idle using Unreal Engine 5's AI Behaviour Tree system.
    - Created custom AI tasks and Decorators to implement robust AI behaviour, allowing Otter to intelligently react to its environment while maximizing responsiveness.
    - My most significant contributions to Otter's behaviour were Fetch and Pet, implementing the Petting mechanic completely independently within Unreal and implementing all of Otter's fetch behaviour.
    - The majority of my work was completed in UE5 Blueprints, although I also wrote C++ helper functions that could be called from Blueprints, primarily for Otter's follow behaviour.
2. Animation State Machine & Locomotion
    - Imported all animations created by the animators from Maya into Unreal Engine 5.
    - Performed IK retargeting between multiple skeletal meshes to ensure animations from Maya appear as expected in Unreal Engine 5.
    - Designed and implemented the Animation State Machine to communicate with Otter's AI to ensure smooth animation transitions and realistic behaviour.
    - Defined dog locomotion logic in the Animation Blueprints to correctly pace dog movement speed to sync with locomotion animations and used root motion for any available animation montages.
3. Level Design & Playtesting
    - Assisted in the development of grey-boxed test levels to debug and test Otter's movements and behaviour and to facilitate play tests and trailer recordings.
    - Organized two playtest sessions and gathered player feedback on Otter's responsiveness and interactions with the player. This information was used to guide later development.
4. Tooling & Team Collaboration
    - All assets, Blueprints files, and C++ files were submitted to Tidepool's main Perforce depot.
    - Tracked tasks using Jira as well as in-person Kanban boards to organize development. Progress was documented during bi-weekly agile sprint retrospectives.
    - Participated in daily Scrum stand-ups, Sprint Planning, and Client-Review meetings. Adapted features based on client input (e.g., adjusting Otter's follow radius for better visibility).

Screenshots:



Dog Petting Animations and Interaction



Dog Companion with Player



Dog Locomotion Following Player



Dog Fetching a Stick

Personal Projects

OpenGL/C++ 3D Rasterizer

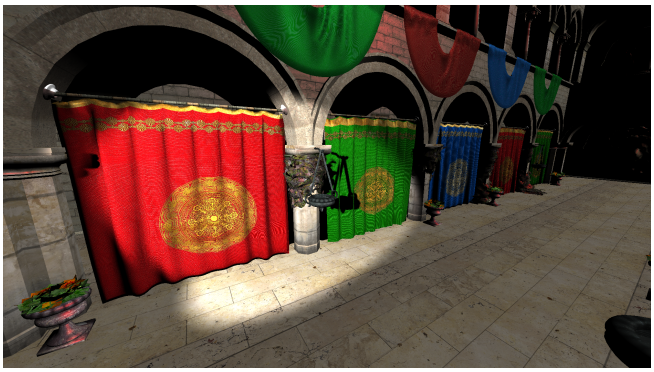
Technologies: C++, OpenGL, Dear ImGui
Project Site: https://github.com/rsolis096/OpenGL-Project

This project is a custom 3D renderer built using C++ and OpenGL as a platform for studying real-time graphics programming. It supports both forward and deferred rendering, with Blinn-Phong lighting, multiple light types, real-time shadow mapping, and model loading through Assimp. A Dear ImGui interface provides runtime access to scene objects, lighting parameters, rendering settings, and performance information. I continue to build on the project as my interests grow and am currently developing a separate networking prototype using Boost.Asio, with the long-term goal of supporting multiplayer functionality in the renderer.

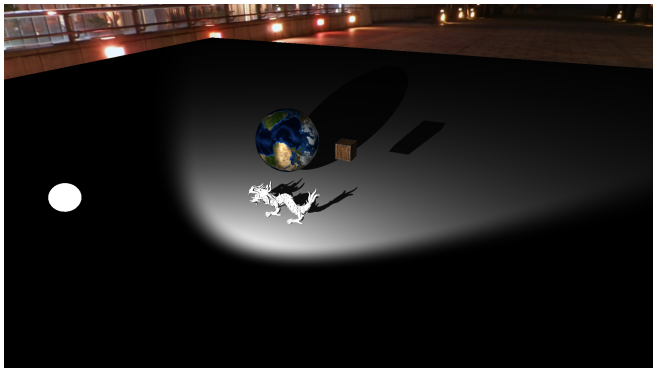
Features:

1. Rendering Pipeline
    - Implemented both forward and deferred rendering pipelines, allowing scenes to be rendered through either direct lighting passes or a geometry buffer.
    - Built a G-buffer containing scene position, normal, albedo, and material data for use during deferred lighting.
    - Implemented Blinn-Phong shading with directional, point, and spot lights across the forward and deferred pipelines.
    - Integrated real-time shadow mapping and Percentage Closer Filtering to produce softer shadow edges.
    - Used OpenGL queries to profile rendering performance while processing over 10 million triangle primitives per frame across geometry and shadow passes.
2. Objects and Model Loading
    - Added Assimp-based model loading with support for OBJ, FBX, glTF, 3DS, and STL formats, allowing import of complex meshes at runtime.
    - Implemented a resource manager to handle loading textures (diffuse, specular maps) using stb-image for sky maps or object/model textures.
    - Structured scene graph data so that both built-in primitives (cubes, spheres, planes) and imported models could be rendered via the same GPU buffers and draw calls.
    - Enabled real-time mesh transformations (translation, rotation, scaling) through uniform updates in the shader, facilitating quick scene edits and debugging.
3. Graphical User Interface
    - Built a Dear ImGui panel with 30+ widgets such as object list, light list, transform/color pickers, shadow controls, and real-time FPS and object counters, providing feedback and debugging information.
    - Exposed camera controls and shader parameters (e.g., ambient/diffuse/specular values, shadow bias) as sliders and color pickers to allow interactive tuning without recompilation.
    - Organized GUI sections into collapsible folders (e.g., “Scene Objects,” “Light Settings,” “Renderer Options”) so users could easily focus on one category at a time.

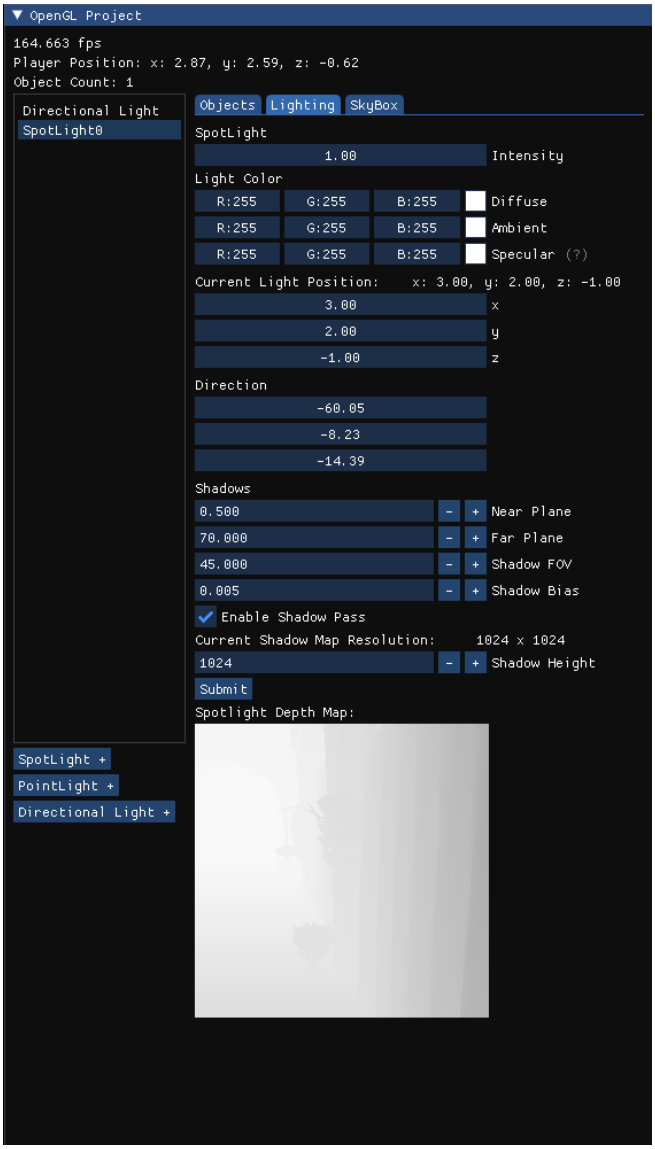
Screenshots:



Textures + Lights



Textured models with shadow maps



Light GUI Panel



Object GUI Panel

C++/SDL Flappy Bird Clone – PS Vita Port

Technologies: VitaSDK, C++, SDL2
Project Site: https://github.com/rsolis096/Flappy_Vita

For fun, I ported a Windows-based Flappy Bird clone (originally written in SDL2 and C++) to the PlayStation Vita. Lacking extensive VitaSDK documentation, I had to heavily rely on community forums, reading VitaSDK examples, and consulting more experienced developers online to configure a working build environment for the Vita. As this was a port, this project revealed many performance bottlenecks that were hidden by more powerful desktop hardware, particularly due to the Vita's limited RAM and VRAM. This required me to refactor the rendering code to achieve a stable 60 FPS on the Vita. These optimizations included: improved texture loading to reduce RAM usage, font caching, and implementing touch and button controls. The repository includes application files for both Windows and the Vita, with installation instructions.

Features:

1. **VitaSDK & SDL2 Integration**
   - Used VitaSDK's native SDL2 support for loading textures and audio on-device.
2. **Performance Optimization**
   - Profiled CPU usage to identify costly draw calls (rendering and text updates).
   - Reduced font memory by using SDL_FontCache and adjusting text sizing.
   - Refactored UI layout and game logic for horizontal gameplay (original was vertical).
3. **Input & Audio Handling**
   - Implemented dual controls: touchscreen tap or `X` button to flap.
   - Integrated SDL2_mixer with VitaSDK callbacks for flapping, scoring, and collision sound effects.
4. **Cross-Platform Build & Deployment**
   - Wrote CMakeLists and Makefiles to include all libraries and assets for Vita builds.
   - Packaged sprites, fonts, and audio into a VPK file for easy installation on any PS Vita.



Flappy Bird clone running on a PS Vita.

C#/Unity Global Game Jam Submission

Technologies: Unity, C#
Project Site: https://github.com/rsolis096/GlobalGameJam2026

In February 2026, two teammates and I participated in Global Game Jam, where we developed a game in two days around the theme "Mask". We created a physics-based balancing game about two kids stacked in a trench coat attempting to pass as an adult while moving through a konbini corner store. The player has to carefully balance movement to avoid toppling over while trying to complete their objective.

I took on the role of main programmer, working alongside two technical artists. I implemented all core game systems myself, including the player controller, physics-based balancing, scene loading, and win/lose conditions. We presented the project in person in Victoria, BC to an audience of local game developers and received very positive feedback. Seeing people enjoy a game built under such tight constraints was very rewarding and reinforced my passion for game development.

Because this was a two-day game jam, the technical scope was intentionally small. However, implementing the core systems from scratch under a strict deadline was a significant challenge and a valuable learning experience in rapid prototyping.

Screenshots:



Konbini Market
Modelled by Rita Chu



Player Character
Modelled by Jasmine Wei